

Relatório sobre o uso de LLMs

Parte 1:

Todo o trabalho foi feito utilizando o Gemini gratuito no dia 08/06/2026. Ele possuía acesso à internet. As respostas de código foram verificadas compilando e testando o código, além de usar conhecimento prévio de C.

Parte 2:

Primeiro prompt: “Olá, explique o que é a máquina de Turing.”

Resposta: Olá! A Máquina de Turing é um conceito fundamental da ciência da computação. Ela não é uma máquina física (com engrenagens ou circuitos), mas sim um modelo matemático abstrato criado pelo matemático britânico Alan Turing em 1936.

A grande sacada de Turing foi criar um sistema incrivelmente simples que serve para simular a lógica de qualquer computador moderno. Se um problema pode ser resolvido por um algoritmo, uma Máquina de Turing consegue resolvê-lo.

Como ela funciona?

Imagine um dispositivo conceitual composto por apenas quatro partes básicas:

Uma Fita Infinita: Dividida em quadradinhos (células), como uma fita de filme negativa. Cada quadrado pode conter um símbolo (por exemplo, 0, 1 ou estar em branco). Ela funciona como a memória do computador.

Uma Cabeça de Leitura/Escrita: Um mecanismo que fica apontado para um quadrado da fita por vez. Ela consegue ler o símbolo atual, apagar, escrever um novo símbolo e se mover para a esquerda ou para a direita.

Um Registro de Estado: Guarda a "situação" atual da máquina (o contexto). Por exemplo: "Estado A", "Estado B" ou "Estado de Parada" (quando o trabalho termina).

Uma Tabela de Transição (As Regras): O "programa" da máquina. É um conjunto de instruções fixas que diz o que fazer com base no estado atual e no símbolo que está sendo lido.

O Ciclo de Execução

A máquina funciona em um loop muito simples baseado nas regras da tabela:

Se estou no Estado A e leio o símbolo 1:

Escreva o símbolo 0.

Mova a cabeça para a Direita.

Mude para o Estado B.

Por que ela é tão importante?

1. Definiu o que é um Computador

Antes de existirem microchips, Turing provou matematicamente o que significa "computar" algo. Ele mostrou que computação nada mais é do que manipular símbolos em uma fita seguindo regras simples.

2. A Máquina Universal de Turing (UTM)

Turing foi além: ele percebeu que uma Máquina de Turing poderia ser programada para ler a tabela de regras de outra máquina de Turing na própria fita e imitá-la. Esse conceito da Máquina Universal é exatamente o que o seu computador ou smartphone é hoje: um único hardware que roda qualquer software (navegador, jogos, editores de texto).

3. Os Limites da Computação

Ao criar esse modelo, Turing conseguiu provar que existem problemas matemáticos que nenhum computador, por mais poderoso que seja, jamais conseguirá resolver. O exemplo mais famoso é o Problema da Parada (Halting Problem), que prova ser impossível criar um programa perfeito que determine se qualquer outro programa vai terminar de rodar ou vai ficar em um loop infinito.

Em resumo, a Máquina de Turing é o blueprint espiritual da computação. Toda a tecnologia digital que usamos hoje — do algoritmo do seu feed de redes sociais às inteligências artificiais — opera sob os mesmos princípios lógicos que Alan Turing colocou no papel quase um século atrás.

Segundo prompt: “Considere que você vai dar uma palestra sobre a máquina de Turing e sua importância para a computação para os calouros de ciência da computação de uma universidade. Explique os conceitos fundamentais e mais relevantes sobre o tema.”

Resposta: Parabéns pela iniciativa! Falar para calouros é uma oportunidade incrível. Eles estão cheios de entusiasmo, mas ainda não têm o jargão técnico. O segredo aqui é focar no impacto conceitual e na narrativa histórica, sem deixar o rigor de lado, mas evitando que eles se percam em formalismos matemáticos densos logo no primeiro mês de faculdade.

Aqui está um roteiro estruturado para a sua palestra, pensado para prender a atenção e construir uma base sólida.

1. A Introdução: O Contexto Histórico (O "Gancho")

Antes de explicar o que é a máquina, explique por que ela surgiu.

O Cenário (Anos 1930): Computadores eletrônicos não existiam. "Computador" era um cargo de trabalho — pessoas (geralmente mulheres) contratadas para fazer contas exaustivas à mão.

O Desafio de Hilbert: O matemático David Hilbert propôs um desafio: existiria um método mecânico definitivo (um algoritmo) capaz de provar se qualquer afirmação matemática era verdadeira ou falsa? (O chamado Entscheidungsproblem).

O Herói: Alan Turing, aos 24 anos, resolveu responder a isso. Para provar se um "método mecânico" existia ou não, ele precisou primeiro definir, matematicamente, o que significa calcular.

2. O Conceito: O que é a Máquina de Turing?

Explique que ela não é uma máquina de metal e engrenagens, mas sim um modelo matemático abstrato. Use a analogia de uma pessoa fazendo contas em um papel quadriculado de tamanho infinito.

A máquina é composta por apenas quatro elementos incrivelmente simples:

A Fita: Uma fita infinita dividida em células (quadrados). Cada célula pode conter um símbolo (como 0, 1 ou estar em branco). É a memória do nosso computador.

A Cabeça de Leitura/Escrita: Um dispositivo que se posiciona sobre uma célula da fita por vez. Ela pode ler o símbolo atual, apagar, escrever um novo símbolo e se mover uma célula para a esquerda ou para a direita.

O Registro de Estado: A "mente" da máquina. Um indicador de em qual "estado" ela se encontra atualmente (ex: "Estado A: procurando o primeiro zero", "Estado B: somando", "Estado de Parada").

A Tabela de Transição (O Programa): O conjunto de regras rígidas que diz à máquina o que fazer.

Exemplo de regra: "Se você estiver no Estado A e ler o símbolo 1, mude para o Estado B, escreva 0 e mova para a direita."

3. A Grande Revelação: A Máquina de Turing Universal (MTU)

Aqui é onde a cabeça dos calouros vai explodir.

Até então, se você quisesse uma Máquina de Turing para somar, você criava uma máquina com regras de soma. Se quisesse uma para multiplicar, criava outra.

Mas Turing teve um estalo genial: e se criássemos uma máquina cuja fita contivesse não apenas os dados, mas também a descrição das regras de outra máquina?

Isso é a Máquina de Turing Universal.

Ela lê o "programa" na fita e simula o comportamento desse programa com os dados que estão logo adiante.

A Conexão com o Presente: Explique que a MTU é, literalmente, o conceito do computador moderno. Onde a fita é o disco rígido/memória RAM e a tabela de transição universal é a CPU. O seu smartphone é uma Máquina de Turing Universal.

4. A Importância Absoluta para a Computação

Por que os calouros precisam estudar isso? Destaque três pilares:

A Tese de Church-Turing (O Limite do Computável)

Tudo o que um supercomputador moderno, um computador quântico ou uma inteligência artificial consegue calcular, uma Máquina de Turing também consegue (só que muito mais devagar). Ela define a fronteira do que é computável no universo.

O Problema da Parada (O Limite do que Não Podemos Fazer)

Turing provou que existem problemas que nenhum computador jamais conseguirá resolver, não importa quanta memória ou processamento ele tenha.

O exemplo clássico é o Problema da Parada (Halting Problem): é impossível escrever um programa perfeito que analise qualquer outro código e diga, com 100% de certeza, se esse código vai rodar para sempre (loop infinito) ou se vai terminar sua execução.

A Origem da Arquitetura de Computadores

Ao separar o hardware (a máquina física/abstrata) do software (as instruções escritas na fita), Turing inventou o conceito de programação.

5. Conclusão: Mensagem de Inspiração para os Calouros

Feche a palestra conectando o passado ao futuro deles:

"Ciência da Computação não é sobre computadores, assim como a Astronomia não é sobre telescópios. É sobre a resolução de problemas por meio de processos lógicos. Alan Turing moldou o mundo moderno sem ligar um único fio na tomada, apenas usando lógica pura, papel e caneta. O que vocês vão aprender nos próximos quatro anos é como dominar essa mesma lógica para construir o futuro."

Dicas extras para a apresentação:

Visual: Use animações simples (GIFs ou simulações web) de uma Máquina de Turing funcionando (ex: ela invertendo bits de 0 para 1). Ver a cabeça se movendo ajuda a fixar o conceito.

Interação: Pergunte no início se alguém sabe o que é o Problema da Parada ou quem assistiu ao filme "O Jogo da Imitação". Isso quebra o gelo.

Comparação entre as duas respostas: Mesmo tendo sido criado em dois chats diferentes, as respostas foram consideravelmente aproximadas (possivelmente porque o segundo prompt mencionava calouros, então ele simplificou tanto quanto o primeiro prompt). O segundo prompt até foi mais específico, mas talvez o uso do prompt pedindo para ser uma palestra fez com que o Gemini pensasse mais sobre a palestra e menos sobre o conteúdo.

Parte 3:

Primeiro prompt: “Faça um programa em C que peça o nome do usuário e depois printe na tela o nome invertido.”

Código gerado:

```
#include <stdio.h>
#include <string.h>

int main() {
    char nome[100];
    int tamanho, i;

    // Solicita o nome do usuário
    printf("Digite o seu nome: ");

    // fgets é mais seguro que scanf para ler nomes com espaços
    fgets(nome, sizeof(nome), stdin);

    // Remove a quebra de linha (\n) que o fgets armazena no final da string
    nome[strcspn(nome, "\n")] = '\0';

    // Calcula o tamanho da string
    tamanho = strlen(nome);

    // Exibe o nome invertido
    printf("Seu nome invertido: ");
    for (i = tamanho - 1; i >= 0; i--) {
        printf("%c", nome[i]);
    }
    printf("\n");

    return 0;
}
```

Explicação (gerada automaticamente pelo próprio Gemini no mesmo prompt):

Como o código funciona:

`fgets(nome, sizeof(nome), stdin)`: Usamos o `fgets` em vez do `scanf` porque ele consegue ler espaços em branco. Assim, se você digitar "João Silva", o programa não vai parar no primeiro espaço.

`nome[strcspn(nome, "\n")] = '\0'`: O `fgets` captura o "Enter" (`\n`) no final da digitação. Essa linha serve para encontrar esse `\n` e substituí-lo pelo caractere nulo (`\0`), que indica o fim da string. Se não fizermos isso, o programa tentará inverter a quebra de linha também!

O loop `for`: Começamos o contador `i` na posição `tamanho - 1` (que é a última letra válida da string) e vamos decrementando até `0` (a primeira letra), imprimindo caractere por caractere.

Dois testes (feitos manualmente):

Primeiro teste: Entrada: "Caua Muller Henkel". Saída: "lekneH relluM auaC".

Segundo teste: Entrada: "Joao Pedro1122". Saída: "2211ordeP oaoJ".

Análise crítica:

Por mais que o código pedido seja relativamente simples, foi bem implementado mesmo sem ter praticamente nenhuma especificação. A indentação está correta, existem comentários suficientemente descritivos e as variáveis utilizadas possuem nomes fáceis de entender.

Parte 4:

Novo prompt: "Faça um programa em C que peça o nome do usuário e depois printe na tela o nome invertido. Adicione comentários explicativos, dê exemplos de entrada e saída e mencione algumas restrições de implementação."

Código e resposta gerada:

```
#include <stdio.h>
#include <string.h>

int main() {
```

```

// Declaração de um array de caracteres (string) com capacidade para 99 caracteres +
'\0'

char nome[100];
int tamanho, i;
// Solicita o nome do usuário
printf("Digite o seu nome: ");

// fgets é mais seguro que scanf porque evita estouro de memória (buffer overflow)
// Ele lê a linha inteira, incluindo espaços, até que o usuário aperte Enter
fgets(nome, sizeof(nome), stdin);

// O fgets costuma guardar o caractere de nova linha '\n' (o Enter) no final da string.
// O trecho abaixo remove esse '\n' para que ele não seja invertido junto com o nome.
tamanho = strlen(nome);
if (tamanho > 0 && nome[tamanho - 1] == '\n') {
    nome[tamanho - 1] = '\0';
    tamanho--; // Atualiza o tamanho real do nome
}

printf("Nome invertido: ");

// Loop que começa do último caractere válido (tamanho - 1)
// e vai voltando até o primeiro caractere (índice 0)
for (i = tamanho - 1; i >= 0; i--) {
    printf("%c", nome[i]);
}

printf("\n"); // Quebra de linha final

return 0;
}

```

Exemplos de Entrada e Saída

Exemplo 1: Nome Simples

Entrada: Manoel

Saída: Nome invertido: leonaM

Exemplo 2: Nome Composto (com espaços)

Entrada: Ana Beatriz

Saída: Nome invertido: ziraeB anA (**A IA errou o exemplo**)

Restrições de Implementação e Cuidados

Ao lidar com strings em C, existem algumas armadilhas clássicas que você deve ter em mente:

Limite de Caracteres (Estouro de Buffer): O vetor foi definido com tamanho 100. Como o último caractere de uma string em C deve ser obrigatoriamente o terminador nulo (`\0`), o programa consegue ler um nome de no máximo 99 caracteres. Se o usuário digitar algo maior, o `fgets` vai cortar o nome para caber no limite de segurança.

O Caractere Oculto `\n`: Funções de leitura como o `fgets` capturam o "Enter" que o usuário pressiona. Se você não "limpar" esse `\n` (como feito no código acima), o programa tentará invertê-lo, gerando uma quebra de linha errada logo no início do nome invertido.

Problemas com Acentuação (Unicode/UTF-8): Este código funciona perfeitamente para caracteres da tabela ASCII padrão (letras sem acento). Se você digitar um nome com acentos ou caracteres especiais (como André ou João), a saída pode parecer bagunçada ou com símbolos estranhos. Isso acontece porque caracteres acentuados geralmente ocupam mais de 1 byte na memória em sistemas modernos (UTF-8), e a lógica de inverter caractere por caractere acaba "partindo" esses bytes ao meio.

Análise crítica:

Mesmo tendo sido criado em outro chat, o código gerado é praticamente idêntico, mudando apenas os comentários que ficaram um pouco mais descritivos. Talvez para códigos grandes com muitas funções a IA passe a mudar muita coisa e fazer códigos que não funcionem, mas para códigos simples ela se mantém bem consistente e funcional. As restrições trazidas pela IA também são pertinentes.

Parte 5:

Peguei como referência a segunda resposta.

1. A resposta parece estar suficientemente correta, sem nenhum erro perceptível ou alucinação.
2. Faltou apenas mencionar na explicação da máquina de Turing que também existe em sua definição o alfabeto utilizado (quais símbolos a máquina aceita).
3. Verifiquei a resposta com meus conhecimentos e pesquisa em sites.
4. Não me pareceu que o Gemini agiu com excesso de confiança em nenhum momento, por mais que ele geralmente fala como se todas as suas informações estivessem corretas.
5. Poderia ser retirado o pedido sobre a palestra e talvez ter pedido para o Gemini desenvolver mais o assunto, já que em ambos os casos ele pareceu falar muito superficialmente sobre os assuntos, mesmo falando que era para alunos da universidade.

Parte 6:

Uso responsável de LLMs:

1. Não devemos copiar respostas sem revisão porque LLMs não são geradores de textos corretos, são simplesmente geradores de texto. Eles podem simplesmente alucinar do nada e tirar conclusões totalmente incorretas sobre assuntos importantes, tornando indispensável a revisão de tudo que é tirado como conclusão usando LLMs.
2. É importante ter salvo os prompts utilizados para entender o que foi utilizado como referência pelo LLM, servindo de base para entender o que talvez não foi especificado, ou o que o modelo pode ter criado a mais sem necessidade e que pode ter comprometido o resultado final.
3. Os LLMs são extremamente versáteis no estudo, pois conseguem responder perguntas específicas com alta velocidade e grande precisão. Mesmo sendo necessário haver um senso crítico na hora de usar suas respostas como referência, a maioria das respostas consegue ser bem precisa. O uso de LLMs pode reduzir em muito o tempo de pesquisa que seria gasto para resolver um problema ou dificuldade específica do aluno (além de que muitas vezes as informações na internet também podem não ser confiáveis).
4. Os LLMs atrapalham quando a pessoa os usa sem a vontade verdadeira de aprender e acaba usando suas respostas sem ter nenhum senso crítico ou pensamento profundo em relação à resposta dada. Quando se usa a IA não como ferramenta de apoio, mas como uma máquina que pensa tudo por você, o aluno não absorve nada para si.
5. Significa haver sempre um senso crítico e um pensamento sobre cada resposta dada pelo LLM. Se é pedido um código para um exercício, por exemplo, é ideal que cada linha de código seja analisada, buscando entender o algoritmo por trás. Mesmo em outras áreas é importante que as informações não sejam copiadas diretamente, mas que a pessoa tente criar seu próprio texto e resposta usando como base aquilo dito pela IA.

Reflexão final:

1. Usando apenas prompts simples não foi possível testar os limites dos LLMs nem levar eles a cometer grandes erros, mas foi possível perceber como usando prompts pouco descritivos o LLM tomava liberdade para explicar e acrescentar coisas à sua própria vontade, mesmo sem ter sido pedido.
2. Para prompts simples influenciou muito pouco o resultado. Senti que a maior diferença foi nos prompts dos códigos, em que com pouca descrição o Gemini tomava muita liberdade para explicar o que queria, enquanto quando foi especificado o que ele precisava responder ele passou a responder apenas o que foi pedido.
3. Sinto que ela foi mais útil na parte de gerar código, pois geralmente sinto que essas explicações de LLMs podem acabar sendo muito rasas quando não é perguntado nada muito específico. Também preferi as respostas sobre código, que me pareceram mais técnicas e corretas.

4. Eu fui atrás de buscar mais sobre a máquina de Turing, porque não é um conhecimento que eu domino, mas o Gemini não parece ter errado até onde percebi. O código sempre é bom testar para ver se está correto e nesse caso tudo funcionou de primeira, além das explicações estarem bem completas.
5. Eu utilizaria essas ferramentas em disciplinas futuras, porque acabam solucionando a maioria das minhas dúvidas rapidamente sem precisar ficar procurando em diversos sites diferentes. Entretanto, sempre é necessário ter bom senso para analisar o que o modelo pode estar dizendo e ignorar aquilo que parece suspeito. Além de que é ideal usar os LLMs apenas para tirar dúvidas e resolver problemas onde não se está conseguindo resolver sozinho, sempre tendo a noção de primeiro entender o que ele respondeu e só então aplicando esse conhecimento.